

The Robertson Chemical Kinetics Problem: Mathematical Derivation and Numerical Analysis

Mathematical derivation document: model, methods, stability, stiffness, convergence, and Newton theory

Covers items 1–12 of the task assignment; each part follows the pattern "theory → prediction → experiment → interpretation"

Contents

1. The Robertson model and its basic properties
2. Three numerical methods: formulas and theoretical order
3. Newton iteration for implicit Euler
4. Convergence theory and experimental criteria
5. Absolute stability of the three methods
6. The Robertson Jacobian
7. The structural zero eigenvalue and initial degeneracy
8. The stiffness ratio and the reduced formulation
9. Step-size restriction of explicit Euler for Robertson
10. Definitions of the four diagnostics
11. Matched-error comparison
12. The Newton tolerance sweep

1 The Robertson model and its basic properties

1.1 The system

The Robertson chemical kinetics problem describes the evolution of the concentrations of three reacting species $A(y_1)$, $B(y_2)$, and $C(y_3)$:

$$\begin{cases} y_1' = -0.04 y_1 + 10^4 y_2 y_3 \\ y_2' = 0.04 y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2 \\ y_3' = 3 \times 10^7 y_2^2 \end{cases} \quad y(0) = (1, 0, 0)^T \quad (1.1)$$

The three equations correspond to the following reaction channels:

Reaction	Step	Rate constant	Time scale
1 (slow)	$A \xrightarrow{k_1} B$	$k_1 = 0.04$	~ 25 s
2 (fast, catalytic)	$B + C \xrightarrow{k_2} A + C$	$k_2 = 10^4$	depends on y_2, y_3
3 (very fast)	$2B \xrightarrow{k_3} B + C$	$k_3 = 3 \times 10^7$	very short

The three rate constants span nine orders of magnitude, from 0.04 to 3×10^7 . A system in which a very fast transient decays while the rest of the solution evolves slowly is what is normally called *stiff*. This is exactly the phenomenon the project asks about: explicit Euler is forced to take tiny steps to keep the fast mode under control, and the question is whether an implicit method can reach the same accuracy at a much smaller total cost.

1.2 The conservation law

Adding the three equations gives

$$y_1' + y_2' + y_3' = (-0.04 y_1 + 10^4 y_2 y_3) + (0.04 y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2) + (3 \times 10^7 y_2^2). \quad (1.2)$$

Term by term: the y_1 terms cancel ($-0.04 + 0.04 = 0$), the $y_2 y_3$ terms cancel (species C is a catalyst and is produced and consumed at the same rate), and the y_2^2 terms cancel (reaction 3 converts $2B$ into $B + C$, so one B is lost and one C is gained; the total is unchanged). Hence

$$\frac{d}{dt}(y_1 + y_2 + y_3) = 0, \quad (1.3)$$

and with $y(0) = (1, 0, 0)^T$,

$$y_1(t) + y_2(t) + y_3(t) = 1 \quad \forall t \geq 0. \quad (1.4)$$

Key point

The sum $y_1 + y_2 + y_3$ is an invariant of the exact solution. The reactions only transfer material between species; they neither create nor destroy it.

1.3 Positivity

As concentrations, y_i must satisfy $y_i(t) \geq 0$ for all t . Positivity is not a consequence of the conservation law. A numerical solution can perfectly preserve $y_1 + y_2 + y_3 = 1$ and still be unphysical, for instance $y_1 = 1.5$, $y_2 = -0.3$, $y_3 = -0.2$. Some explicit schemes overshoot at

large steps and produce negative concentrations even while the sum remains exactly one. Conservation and positivity are therefore *independent* diagnostics, and both have to be reported.

Expected outcome

The conservation identity $(1, 1, 1) \cdot f(y) = 0$ forces the Jacobian to carry a structural zero eigenvalue at all states (see §7). This has consequences for how the stiffness ratio is computed.

What the code team needs to produce

Two scalar diagnostics for the code team: conservation defect

$\max_i |y_1(t_i) + y_2(t_i) + y_3(t_i) - 1|$ and the minimum component $y_{\min} = \min_{i,j} y_j(t_i)$.

How to read the result

A small conservation defect does not imply a small state error. The three diagnostics (conservation, positivity, accuracy) are to be judged separately throughout the project.

2 Three numerical methods: formulas and theoretical order

The three schemes required by the project brief are derived below from Taylor expansions rather than quoted, and each is related to the Robertson system.

2.1 Explicit Euler

Expanding $y(t_n + h)$ about t_n ,

$$y(t_{n+1}) = y(t_n) + h y'(t_n) + \frac{h^2}{2} y''(t_n) + \mathcal{O}(h^3), \quad (2.1)$$

and keeping only the first-order term with $y' = f(t, y)$ gives

$$y_{n+1} = y_n + h f(t_n, y_n). \quad (2.2)$$

The local truncation error is $\tau_n = \frac{h^2}{2} y''(t_n) + \mathcal{O}(h^3)$, i.e. $\mathcal{O}(h^2)$ per step. Over $N = T/h$ steps these accumulate (in the usual Lipschitz setting) into a global error

$$\|y(T) - y_N\| \leq C h. \quad (2.3)$$

Intuitively, h^2 per step times $1/h$ steps gives $\mathcal{O}(h)$ overall. The difficulty for Robertson is not the order: it is that the step size is pinned down by stability. With $|\lambda_{\max}| \approx 3.4 \times 10^3$, explicit Euler needs $h < 2/|\lambda_{\max}| \approx 5.9 \times 10^{-4}$, so integrating to $T = 40$ takes roughly 6.8×10^4 steps even if we only care about the slow dynamics whose time scale is tens of seconds.

2.2 Classical RK4

RK4 matches the Taylor expansion of $\mathbf{y}(t_n + h)$ up to and including the h^4 term, using four function evaluations per step. Its Butcher tableau is

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array} \quad (2.4)$$

with the stage formulas

$$\begin{aligned} k_1 &= f(t_n, y_n), \\ k_2 &= f(t_n + \frac{h}{2}, y_n + \frac{h}{2} k_1), \\ k_3 &= f(t_n + \frac{h}{2}, y_n + \frac{h}{2} k_2), \\ k_4 &= f(t_n + h, y_n + h k_3), \\ y_{n+1} &= y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4). \end{aligned} \quad (2.5)$$

The local error is $\mathcal{O}(h^5)$ and the global error $\mathcal{O}(h^4)$. For Robertson the scheme is still explicit: its stability region is larger than Euler's but bounded, so on the stiff part of the interval it is equally constrained. It becomes the method of choice only when accuracy, not stability, is the binding constraint.

2.3 Implicit Euler

Using a backward difference for $\mathbf{y}'(t_{n+1})$,

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h \mathbf{f}(t_{n+1}, \mathbf{y}_{n+1}), \quad (2.6)$$

which is a nonlinear equation in $\mathbf{y}_{n+1}$ because $\mathbf{f}$ is evaluated at the unknown state. The local truncation error is again $\mathcal{O}(h^2)$ and the global error $\mathcal{O}(h)$. The first order is the same as explicit Euler; the difference lies in the stability properties, discussed in §5.

2.4 Summary

Method	Type	Global order	RHS evals / step	Stability
Explicit Euler	explicit	1	1	conditionally stable
Classical RK4	explicit	4	4	conditionally stable
Implicit Euler	implicit	1	1 per Newton iteration	A-stable

Expected outcome

The theoretical orders are $p = 1$ (explicit and implicit Euler) and $p = 4$ (RK4). These are only observable in the asymptotic regime: h small enough, and for the explicit methods inside the stability region.

What the code team needs to produce

The code team implements the three methods behind one interface and runs step-size sweeps; the convergence test uses at least four successively halved step sizes and compares the observed order with the theory.

How to read the result

Implicit Euler has the same formal order as explicit Euler. The whole point is not accuracy per step but that it can take much larger steps; the fair comparison is therefore cost at matched accuracy (§11), not error at matched step size.

3 Newton iteration for implicit Euler

3.1 The nonlinear equation

At every step implicit Euler must solve

$$y_{n+1} = y_n + h f(t_{n+1}, y_{n+1}), \quad (3.1)$$

which is nonlinear because of the $y_2 y_3$ and y_2^2 terms. It is convenient to define the residual

$$F(z) = z - y_n - h f(t_{n+1}, z) = 0, \quad (3.2)$$

where z is a candidate for y_{n+1} .

3.2 The Newton update

Linearising F about the current iterate $z^{(k)}$,

$$F(z) \approx F(z^{(k)}) + J_F(z^{(k)}) (z - z^{(k)}), \quad (3.3)$$

and setting the linear model to zero gives

$$z^{(k+1)} = z^{(k)} - [J_F(z^{(k)})]^{-1} F(z^{(k)}). \quad (3.4)$$

The Jacobian of F is obtained by differentiating (3.2):

$$J_F(z) = I - h J_f(t_{n+1}, z), \quad (3.5)$$

where J_f is the Jacobian of the Robertson right-hand side (Section 6).

3.2 Solving the linear system rather than inverting

The inverse in (3.4) is never formed explicitly. The Newton step is written as the linear system

$$J_F(z^{(k)}) \delta^{(k)} = -F(z^{(k)}), \quad z^{(k+1)} = z^{(k)} + \delta^{(k)}. \quad (3.6)$$

There are three reasons. First, numerical stability: forming the inverse (for example by Gauss–Jordan) amplifies rounding error, whereas an LU factorisation followed by forward/backward substitution is stable. Second, efficiency: for an $n \times n$ matrix, the LU factorisation costs $\mathcal{O}(n^3)$ once, and each subsequent right-hand side costs $\mathcal{O}(n^2)$; if the Jacobian is reused over several Newton iterations the factorisation is done once. Third, it is simply the standard practice of numerical linear algebra ("solve, don't invert"), and it matters in larger problems even if the 3×3 Robertson system hides the difference.

3.3 Newton tolerance: loose versus tight

The iteration is stopped when the residual is small:

$$\|F(z^{(k)})\| < \text{tol}_{\text{Newton}}. \quad (3.7)$$

If the tolerance is loose (say 10^{-2}), the iteration stops before $z^{(k)}$ has converged, and the algebraic error $\|z^{(k)} - y_{n+1}^*\|$ can be comparable to or larger than the time discretisation error. Then reducing h does not improve the solution, and a convergence-order test fails in a way that is not the method's fault. If the tolerance is very tight (machine precision), the extra iterations are wasted: once the algebraic error is well below the discretisation error, the total error no longer decreases. A practical guideline is

$$\text{tol}_{\text{Newton}} \ll h^{p+1}, \quad (3.8)$$

where p is the order of the method. For implicit Euler ($p = 1$) and $h = 10^{-2}$, this means $\text{tol} \ll 10^{-4}$; 10^{-6} or smaller is safe.

3.5 The two indices

The notation keeps two levels separate: n indexes the time steps ($y_n \approx y(t_n)$), and k indexes the Newton iterations inside one time step ($z^{(0)}$ is the initial guess, usually y_n ; after convergence, $y_{n+1} := z^{(k_{\text{final}})}$).

Expected outcome

Newton's method converges quadratically close to the solution: $\|z^{(k+1)} - z^*\| \leq C\|z^{(k)} - z^*\|^2$. With $z^{(0)} = y_n$ as the initial guess, two to four iterations per step are typical.

What the code team needs to produce

The Newton solver should record the residual norm and the iteration count at every step, report converged/failed status, and use a linear solve (e.g. `np.linalg.solve`) rather than forming the inverse. Tolerance sweep over $10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}, 10^{-12}$ is required (Section 12).

How to read the result

Newton is cheap for Robertson because the system is three-dimensional: $J_F = I - hJ_f$ is a 3×3 matrix, and two to four iterations per step cost almost nothing. This is what makes the implicit step competitive in practice.

4 Convergence theory and experimental criteria

A method is convergent if, on a fixed interval $[0, T]$,

$$\lim_{h \rightarrow 0} \max_{0 \leq n \leq N} \|y_n - y(t_n)\| = 0, \quad (4.1)$$

and for an order- p method the global error behaves as

$$E(h) = \max_n \|y_n - y(t_n)\| \approx C h^p. \quad (4.2)$$

With two step sizes h and $h/2$, the ratio of the errors gives the observed order,

$$p_{\text{obs}} = \frac{\log(E_h/E_{h/2})}{\log 2}. \quad (4.3)$$

Why four step sizes instead of two? Because the true error is $Ch^p + Dh^{p+1} + \dots$, so the clean power law is only seen when the higher-order terms are negligible (h small enough), when rounding error has not yet taken over (h not too small), and, for explicit methods on stiff problems, when h is inside the stability region. Two sizes can accidentally sit in the transition region and give a misleading order. Plotting $\log E$ against $\log h$ over $h, h/2, h/4, h/8$ and looking for a straight segment, whose slope is p_{obs} , is the robust way. For Robertson this matters: the explicit methods are confined to a narrow band of usable step sizes by the stability constraint, while implicit Euler, being A-stable, has a much wider window and the convergence test is cleaner.

Two error definitions are suggested; the code team should fix one and use it throughout:

Quantity	Definition	Use
Terminal error	$\ y_h(T) - y_{\text{ref}}(T)\ _2$	quick check
Max error over the interval	$\max_n \ y_h(t_n) - y_{\text{ref}}(t_n)\ _2$	strict, catches the worst case

Expected outcome

Within the asymptotic regime, $p_{\text{obs}} \approx 1$ for both Euler schemes and $p_{\text{obs}} \approx 4$ for RK4. All step sizes used for the explicit methods must lie in the stability region; otherwise the error blows up and p_{obs} is meaningless.

What the code team needs to produce

Use $h, h/2, h/4, h/8$ at least; record E_{full} ; plot $\log E$ vs $\log h$ and report the slope.

How to read the result

If p_{obs} deviates from the theory, the usual causes are: the asymptotic regime was not reached, the reference solution is not accurate enough, the Newton tolerance is too loose (for IE), or there is an implementation bug. For stiff problems the explicit methods often show a short or absent straight segment because the stability constraint narrows the admissible step sizes.

5 Absolute stability of the three methods

Absolute stability is studied on the scalar test problem $y' = \lambda y$ with $\text{Re } \lambda < 0$, whose exact solution decays. A one-step method applied to this equation takes the form $y_{n+1} = R(z)y_n$, $z = h\lambda$, and is stable if $|R(z)| \leq 1$. The function R is the stability function of the method.

5.1 Explicit Euler

$y_{n+1} = y_n + h\lambda y_n = (1 + z)y_n$, so

$$R_{\text{EE}}(z) = 1 + z, \quad |1 + z| \leq 1. \quad (5.1)$$

The stability region is the unit disk centred at -1 . For real $\lambda < 0$, the condition $|1 + h\lambda| \leq 1$ reduces to $h \leq 2/|\lambda|$.

5.2 Classical RK4

Applying the stages to the linear test problem with $z = h\lambda$ gives, after simplification,

$$R_{\text{RK4}}(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}, \quad (5.2)$$

which is the Taylor polynomial of e^z truncated at degree four. On the negative real axis the stability boundary is $z \approx -2.78$, so $h \lesssim 2.78/|\lambda|$: a modest improvement over Euler, roughly a factor of 1.4, which does not cure stiffness.

5.3 Implicit Euler

$y_{n+1} = y_n + h\lambda y_{n+1}$ solved for y_{n+1} :

$$R_{\text{IE}}(z) = \frac{1}{1 - z}, \quad \left| \frac{1}{1 - z} \right| \leq 1. \quad (5.3)$$

The stability region is the exterior of the disk of radius 1 centred at $z = 1$. In particular it contains the whole left half-plane: writing $z = a + bi$ with $a < 0$, one has $|1 - z|^2 = (1 - a)^2 + b^2 > 1$, so

$|R_{\text{IE}}(z)| < 1$. This is the defining property of *A-stability*.

This is what matters for Robertson. The Jacobian has eigenvalues with large negative real parts, i.e. $z = h\lambda$ far out on the negative axis. Explicit methods require $|z|$ to stay inside a bounded region, forcing h to be tiny. An A-stable method is stable for every $h > 0$ regardless of $|\lambda_{\text{max}}|$, so the step size is limited only by accuracy. This is the theoretical reason why the implicit method can integrate the stiff system at reasonable cost.

Method	$R(z)$	Region	On the real axis	A-stable
Explicit Euler	$1 + z$	disk $ 1 + z \leq 1$	$-2 \leq z \leq 0$	no
RK4	$\sum_{j=0}^4 z^j/j!$	bounded	$\approx -2.78 \leq z \leq 0$	no
Implicit Euler	$1/(1 - z)$	$ 1 - z \geq 1$	$z \leq 0$ or $z \geq 2$	yes

Expected outcome

EE: $h < 5.9 \times 10^{-4}$ for $|\lambda_{\text{max}}| \approx 3.4 \times 10^3$. RK4: slightly larger, $h \lesssim 8.2 \times 10^{-4}$. IE: no stability restriction at all.

What the code team needs to produce

Plot the three stability regions (or one combined figure), and mark the location of $h\lambda$ for the Robertson Jacobian at a few times to confront the prediction with the experiment.

How to read the result

A-stability is not a guarantee of accuracy. It guarantees only that the solution does not blow up; the error of implicit Euler remains $\mathcal{O}(h)$. The implicit method wins because it can use the largest step allowed by the accuracy target, whereas the explicit methods must use the smallest step forced by stability.

6 The Robertson Jacobian

With

$$\begin{aligned}
 f_1(y) &= -0.04 y_1 + 10^4 y_2 y_3, \\
 f_2(y) &= 0.04 y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2, \\
 f_3(y) &= 3 \times 10^7 y_2^2,
 \end{aligned} \tag{6.1}$$

each entry $\partial f_i / \partial y_j$ is computed directly:

$$\begin{aligned}
\frac{\partial f_1}{\partial y_1} &= -0.04, & \frac{\partial f_1}{\partial y_2} &= 10^4 y_3, & \frac{\partial f_1}{\partial y_3} &= 10^4 y_2, \\
\frac{\partial f_2}{\partial y_1} &= 0.04, & \frac{\partial f_2}{\partial y_2} &= -10^4 y_3 - 6 \times 10^7 y_2, & \frac{\partial f_2}{\partial y_3} &= -10^4 y_2, \\
\frac{\partial f_3}{\partial y_1} &= 0, & \frac{\partial f_3}{\partial y_2} &= 6 \times 10^7 y_2, & \frac{\partial f_3}{\partial y_3} &= 0.
\end{aligned} \tag{6.2}$$

The entry $\partial f_2 / \partial y_2$ deserves a comment: the factor $-6 \times 10^7 y_2$ comes from $\partial(-3 \times 10^7 y_2^2) / \partial y_2 = -6 \times 10^7 y_2$, i.e. the quadratic term in reaction 3. It is the largest entry of the matrix, and it is the direct source of stiffness: as soon as $y_2 > 0$, the Jacobian acquires a large eigenvalue.

Assembling the entries:

$$J_f(y) = \begin{pmatrix} -0.04 & 10^4 y_3 & 10^4 y_2 \\ 0.04 & -10^4 y_3 - 6 \times 10^7 y_2 & -10^4 y_2 \\ 0 & 6 \times 10^7 y_2 & 0 \end{pmatrix}. \tag{6.3}$$

As a check, each column sums to zero:

- column 1: $-0.04 + 0.04 + 0 = 0$;
- column 2: $10^4 y_3 - 10^4 y_3 - 6 \times 10^7 y_2 + 6 \times 10^7 y_2 = 0$;
- column 3: $10^4 y_2 - 10^4 y_2 + 0 = 0$.

This is the matrix version of the conservation law, and it is what produces the structural zero eigenvalue (Section 7).

Expected outcome

The entry $-6 \times 10^7 y_2$ is the direct source of the stiffness. At $t = 0$ one has $y_2 = 0$ and the system is not stiff; as y_2 grows, the stiffness ratio becomes large.

What the code team needs to produce

The code should implement `robertson_jacobian(t, y)`, verify $f(0, [1, 0, 0]) = [-0.04, 0.04, 0]$, and evaluate the Jacobian and its eigenvalues along the reference trajectory.

How to read the result

The $6 \times 10^7 y_2$ term should be shown explicitly in the report: it is the sensitivity of the reaction 3 rate with respect to y_2 , and it is where the "rigidity" of the system lives.

7 The structural zero eigenvalue and initial degeneracy

From Section 6, the rows of J_f sum to zero, equivalently $(1, 1, 1) J_f(y) = 0$ for all y . The vector $(1, 1, 1)^T$ is therefore a left eigenvector of J_f^T for the eigenvalue 0. Since J_f and J_f^T have the same

spectrum, the full 3×3 Jacobian always has a zero eigenvalue. This zero is *structural*: it holds at every state, and it is a direct algebraic consequence of the conservation law. Physically, the dynamics live in the plane $y_1 + y_2 + y_3 = 1$, and there is no evolution in the normal direction; the zero eigenvalue corresponds to that direction.

At the initial point $y(0) = (1, 0, 0)$ the Jacobian reads

$$J_f((1, 0, 0)^T) = \begin{pmatrix} -0.04 & 0 & 0 \\ 0.04 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad (7.1)$$

and $\det(J_f - \lambda I) = -\lambda^2(0.04 + \lambda)$ gives the eigenvalues

$$\lambda = -0.04, 0, 0. \quad (7.2)$$

There are two zeros. One is the structural zero just discussed. The other is a *degeneracy of the initial state*: at $y_2 = y_3 = 0$, both reaction 2 and reaction 3 have zero rate, and the corresponding columns of the Jacobian vanish. This extra zero is not structural, and it splits into a nonzero eigenvalue as soon as y_2 leaves zero. The distinction matters: only one of the two zeros is guaranteed by the conservation law.

Two consequences for the analysis:

- The stiffness ratio is computed from the nonzero eigenvalues only (Section 8). Putting the structural zero in the denominator gives ∞ , which carries no information.
- At $t = 0$ there is only one nonzero mode, so the ratio is degenerate; the eigenvalue and stiffness-ratio plots should start from the first positive logarithmic time rather than from $t = 0$.

Expected outcome

The eigenvalues at the initial point are $\{-0.04, 0, 0\}$. A threshold $|\lambda| < 10^{-10}$ is used to identify the structural zero numerically, since floating point arithmetic will not produce an exact zero.

What the code team needs to produce

Verify the eigenvalues at $y(0)$; compute the eigenvalues along the reference trajectory; exclude the structural zero via the threshold; start the plots at the first positive log-time point.

How to read the result

The structural zero is a property of the model, not a bug. The 2×2 reduced Jacobian of the next section avoids the issue entirely and is the recommended formulation for the main report, with the full Jacobian in the appendix.

8 The stiffness ratio and the reduced formulation

Stiffness is a statement about time scales. The fast mode has relaxation time $1/|\lambda_{\max}|$ and the slow mode $1/|\lambda_{\min}|$; when their ratio is large, a solver must resolve the fast decay to remain stable, while the dynamics of interest evolves on the slow scale. The quantitative measure is the stiffness ratio

$$S(t) = \frac{\max_{\lambda_j \neq 0} |\operatorname{Re} \lambda_j(t)|}{\min_{\lambda_j \neq 0} |\operatorname{Re} \lambda_j(t)|}. \quad (8.1)$$

The structural zero eigenvalue must be excluded (Section 7), since it would make the denominator zero for every t .

Instead of excluding it numerically, one can remove it algebraically. Using the invariant $y_3 = 1 - y_1 - y_2$ in the first two equations,

$$\begin{aligned} y_1' &= -0.04 y_1 + 10^4 y_2 (1 - y_1 - y_2), \\ y_2' &= 0.04 y_1 - 10^4 y_2 (1 - y_1 - y_2) - 3 \times 10^7 y_2^2, \end{aligned} \quad (8.2)$$

and differentiating, the reduced 2×2 Jacobian is

$$\tilde{J}(y_1, y_2) = \begin{pmatrix} -0.04 - 10^4 y_2 & 10^4(1 - y_1 - 2y_2) \\ 0.04 + 10^4 y_2 & -10^4(1 - y_1 - 2y_2) - 6 \times 10^7 y_2 \end{pmatrix}. \quad (8.3)$$

Its two eigenvalues are exactly the two nonzero eigenvalues of the full Jacobian: the constraint $y_1 + y_2 + y_3 = 1$ restricts the dynamics to a plane, the structural zero corresponds to the normal direction, and $\tilde{J}$ is the restriction of J_f to that plane.

Expected outcome

With the reduced Jacobian there is no zero eigenvalue to filter. As y_2 grows, the two eigenvalues separate: one becomes large (driven by $6 \times 10^7 y_2$), the other stays small (driven by 0.04), and $S(t)$ becomes large.

What the code team needs to produce

Implement $\tilde{J}$, compute its two eigenvalues along the trajectory, plot them (log time axis) together with $S(t)$. Put the full Jacobian in the appendix.

How to read the result

Both formulations give the same nonzero spectrum; the reduced version is just cleaner to present. Use the threshold $|\lambda| < 10^{-10}$ for numerical filtering whenever the full Jacobian is used.

9 Step-size restriction of explicit Euler for Robertson

For real $\lambda < 0$ the stability condition of Euler reduces to $h \leq 2/|\lambda|$, and the most restrictive mode is the one with the largest $|\lambda|$:

$$h_{\text{stab}} = \frac{2}{|\lambda_{\text{max}}(t)|}. \quad (9.1)$$

Along the reference trajectory, at $t \approx 40$ the reduced Jacobian has $|\lambda_{\text{max}}| \approx 3.39 \times 10^3$, hence

$$h_{\text{stab}} \approx \frac{2}{3.39 \times 10^3} \approx 5.9 \times 10^{-4}, \quad (9.2)$$

so integrating over $[0, 40]$ requires at least about 6.8×10^4 steps for stability alone.

It must be emphasised that this is a *local, frozen-Jacobian* estimate. It comes from the linear test problem $y' = \lambda y$ and assumes the Jacobian is constant during the step. The Robertson system is nonlinear, its eigenvalues change over time (at $t = 0$, $|\lambda_{\text{max}}| = 0.04$, a hundred thousand times smaller than at $t = 40$), and nonlinear interactions can move energy between modes in a way the linear analysis cannot see. The value 5.9×10^{-4} is therefore a diagnostic guide, not a theorem; it has to be confirmed empirically.

A step-size sweep around $h \approx 5.9 \times 10^{-4}$ should reveal a transition through four regimes:

Step size	Expected behaviour	Diagnostics
$h \ll 5.9 \times 10^{-4}$	stable and accurate	E_{full} small, $y_{\text{min}} \geq 0$
h slightly below the threshold	stable but inaccurate	no blow-up, large E_{full}
h slightly above the threshold	negative concentrations	$y_{\text{min}} < 0$
$h \gg 5.9 \times 10^{-4}$	unstable/oscillating	divergence

Expected outcome

A smooth transition is expected: accurate $\rightarrow$ stable-but-wrong $\rightarrow$ negative $\rightarrow$ divergent. The negative-concentration regime is particularly instructive because it separates "bounded" from "physical".

What the code team needs to produce

Sweep $h \in \{10^{-5}, 10^{-4}, 3 \times 10^{-4}, 5 \times 10^{-4}, 6 \times 10^{-4}, 7 \times 10^{-4}, 10^{-3}, 10^{-2}\}$; record E_{full} , y_{min} , E_{cons} for each.

How to read the result

The experiment is the empirical check that complements the linear prediction. The value 5.9×10^{-4} is not exact because the system is nonlinear; the sweep locates the actual threshold. Implicit Euler, being A-stable, shows none of these transitions.

10 Definitions of the four diagnostics

With several people working on the same project, different definitions of "error" or "stability" make results hard to compare. The following four quantities are the agreed standard for the whole team.

1. **Full error** $E_{\text{full}} = \max_i \|y_{\text{num}}(t_i) - y_{\text{ref}}(t_i)\|_2$: the worst deviation over the whole history.
This is the quantity used for convergence tests and cost comparisons.
2. **Terminal error** $E_{40} = \|y_{\text{num}}(40) - y_{\text{ref}}(40)\|_2$: a single-number check at the end point, also the quantity reported in the Newton sweep because $y_2(40)$ is particularly sensitive.
3. **Conservation defect** $E_{\text{cons}} = \max_i |y_1(t_i) + y_2(t_i) + y_3(t_i) - 1|$: how badly the invariant is violated.
4. **Minimum component** $y_{\min} = \min_{i,j} y_j(t_i)$: negative value indicates non-physical concentrations.

Three independent criteria are used in the project:

Criterion	Condition	Meaning
Stable	solution bounded on $[0, T]$	no divergence
Non-negative	$y_{\min} \geq -\varepsilon$	no non-physical negative state
Accurate	$E_{\text{full}} < \text{tol}_{\text{target}}$	close to the reference

None of these implies another. "Did not blow up" says nothing about accuracy; a well-conserved sum says nothing about the individual components. All four are reported for every experiment.

Expected outcome

Conservation, positivity, and accuracy are independent and must be judged separately throughout the report.

What the code team needs to produce

The four diagnostics are computed for every run and tabulated per method/step size.

How to read the result

Be prepared to give a concrete example, e.g. explicit Euler near the stability threshold keeping the sum to machine precision while y_2 is in error by 100%.

11 Matched-error comparison

Comparing methods at the same step size is not fair for two reasons. First, the methods have different orders: at the same h , RK4 is always far more accurate than the two Euler schemes, and implicit Euler looks bad because it does more work per step for the same order as explicit Euler. Second, for a stiff problem the step size of the explicit methods is forced by stability, not by accuracy. Comparing all three at a tiny h hides the one property that makes implicit Euler interesting, namely that it may take large steps.

The fair comparison is *matched accuracy*:

1. pick target errors, say $E_{\text{full}} \approx 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}$;
2. for each method find the step size that reaches each target (for EE and RK4 this is subject to the stability constraint);
3. compare the cost of reaching that accuracy.

The natural cost measure is the number of RHS evaluations: 1 per step for EE, 4 per step for RK4, and k_n (the number of Newton iterations) per step for IE, plus the cost of the Jacobian and the linear solves, which is negligible for a 3×3 system. For IE the Newton iteration count is reported separately.

Expected outcome

At low to moderate accuracy targets, implicit Euler is expected to need far fewer RHS evaluations than explicit Euler, because its step is limited by accuracy, while Euler's is limited by stability. Whether RK4 can compete depends on whether the accuracy requirement allows a step size inside its stability region.

What the code team needs to produce

For each accuracy target, record the total RHS evaluations (and Newton iterations for IE) and draw a cost-vs-accuracy log-log plot.

How to read the result

If someone claims a method is better because "it did not blow up", the response is: boundedness is not accuracy. The plot of cost against error is the only fair comparison.

12 The Newton tolerance sweep

Fix a reasonable step size h for implicit Euler and vary the Newton tolerance over

$$\text{tol}_{\text{Newton}} \in \{10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}, 10^{-12}\}.$$

Record, for each tolerance: $y_2(40)$, E_{full} , E_{cons} , and the total number of Newton iterations.

The total error of one implicit step splits into

$$E_{\text{total}} = E_{\text{disc}} + E_{\text{alg}}, \quad (12.1)$$

where $E_{\text{disc}} \sim Ch$ is the time-discretisation error (fixed once h is fixed) and $E_{\text{alg}} \sim \text{tol}$ is the algebraic error from the incomplete Newton solve. Two regimes follow:

- If the tolerance is too loose, $E_{\text{alg}} \gg E_{\text{disc}}$, the total error is dominated by Newton. Reducing h no longer reduces the error, so a convergence test fails even though the implementation is correct.
- If the tolerance is tight enough, $E_{\text{alg}} \ll E_{\text{disc}}$, the total error saturates at E_{disc} . Further tightening only wastes iterations.

Why is $y_2(40)$ the sensitive probe? The equilibrium value of y_2 is of order 10^{-5} , while y_1, y_3 are $\mathcal{O}(1)$. Any absolute error therefore shows up as a large relative error in $y_2(40)$, making it the first place where a too-loose Newton tolerance shows. The expected behaviour is a plot that changes at first and then forms a plateau: the plateau value is the accuracy limit of the step size used, and the tolerance at the transition is the recommended one (about h^2 , or one order tighter).

Expected outcome

$y_2(40)$ changes with the tolerance until the algebraic error drops below the discretisation error, then becomes constant. The recommended Newton tolerance is the transition point, roughly $\text{tol} \approx h^2$ or slightly smaller.

What the code team needs to produce

Run IE with fixed h (e.g. 10^{-2} or 10^{-3}) over the five tolerances; record $y_2(40)$, E_{full} , E_{cons} , and total iterations; plot $y_2(40)$ vs tol and iterations vs tol.

How to read the result

This connects directly to the cost comparison: a tolerance chosen too tight would make the implicit method look bad in Section 11. Choosing the plateau tolerance gives the implicit method the efficiency it actually has.

Summary: theory and experiment

Theory	Experiment	Diagnostics
conservation §1	check E_{cons} after every run	E_{cons}
positivity §1	check y_{min}	y_{min}
orders EE/RK4/IE §2	convergence test	p_{obs}

Newton convergence §3	iteration/residual log	iterations
asymptotic regime §4	$\log E$ vs $\log h$	linear segment slope
stability regions §5	regions + $h\lambda$ points	plot
Jacobian §6	verify at $y(0)$	eigenvalues
structural zero §7	verify $\{-0.04, 0, 0\}$ at $t = 0$	eigenvalues
$S(t)$ §8	$\lambda_1(t), \lambda_2(t), S(t)$	$S(t)$
Euler threshold §9	sweep around 5.9×10^{-4}	$y_{\min}, E_{\text{full}}$
diagnostics §10	report four quantities	all four
matched error §11	cost vs accuracy plot	RHS evals
Newton tolerance §12	$y_2(40)$ vs tol	$y_2(40)$

This document covers items 1–12 of the task assignment for mathematical member. All derivations are placed in the Robertson context, and each section follows theory $\rightarrow$ prediction $\rightarrow$ experiment $\rightarrow$ interpretation.